HELPNET

Course End Project Report

Document status: Draft for team and supervisor review **Prepared:** 2026-11-03 **Repository:** `nidhisamiaanjum-create/HELPNET` **Branch at audit start:** `samia` **Baseline commit at audit start:** `e1523e3`

> ***Submission note:*** *This report is based on the repository and test evidence available for this audit. No teacher template, SRS, SDS, approved project proposal, or complete team record was present. Bracketed fields require team confirmation. This is not a claim of stakeholder acceptance or production deployment.*

Cover information requiring confirmation

- **Course / department:** [Add official course and department]
- **Institution:** [Add institution]
- **Supervisor:** [Add supervisor name and title]
- **Team members and student IDs:** [Add verified roster]
- **Contribution allocation:** [Add each member's confirmed work]
- **Budget / funding:** [Add verified budget or state "No project budget"]
- **Final submission date:** [Confirm]

Abstract

HELPNET is a web-based community services platform developed with Django. It brings together account/profile management and community workflows for volunteering, blood donation, goods exchange, health questions, farmer marketplace listings, waste pickup, ratings, notifications, identity review, and content moderation. The backend exposes authenticated APIs and the user interface uses Django-served HTML templates with JavaScript.

This report summarizes the implementation and the evidence available in the repository. During the audit, several defects were corrected, including insecure defaults, unsafe verification-page rendering, a volunteer certificate route mismatch, inconsistent empty rating averages, and public blood-request documents. The final backend test run passed 115 tests. Django checks and migration-drift checks also passed under the SQLite test configuration.

Browser end-to-end testing, JavaScript build/syntax checks, and live MySQL validation could not be completed in this environment. Historical Git content also still includes previously tracked private uploads; the current index was cleaned without rewriting shared history. The project is therefore presented as an implemented prototype with verified backend coverage and explicit deployment and acceptance work remaining.

1. Introduction

1.1 Background

Community support tasks can be spread across separate channels and organizations. HELPNET provides one web application for several community service workflows, including finding volunteer opportunities, managing blood requests, sharing goods, asking health questions, posting produce, arranging waste pickup, and communicating through notifications and messages.

1.2 Problem statement

The project addresses the need for a shared service platform where users can discover and participate in community services while authorized coordinators and administrators manage requests, verification, and moderation. The repository does not contain a formally approved problem statement or SRS; this description summarizes the behavior implemented in the code.

1.3 Objectives

1. Provide user registration, authentication, user profiles, and profile

switching.

2. Provide APIs and pages for community-service workflows.
3. Enforce authentication, ownership, role, and administrative permissions.
4. Support private document submission and authorized review/download.
5. Provide notifications, ratings, reporting, and moderation capabilities.
6. Exercise backend behavior with automated tests and document known gaps.

1.4 Scope

The audited source contains Django applications for users, blood, goods, health, farmer, waste, volunteer, ratings, reports, notifications, identity verification, and shared/common page routing. The frontend consists of Django templates, CSS, and vanilla JavaScript. The normal application database is configured for MySQL; tests select SQLite.

1.5 Out of scope or not evidenced

No evidence was found for a separate mobile application, automated backup service, active Cloudinary storage integration, full production deployment, formal acceptance sign-off, or independent event/gathering model. These capabilities are not claimed as completed.

2. Requirements and project evidence

2.1 Implemented capability groups

- **Accounts:** Registration, login, password reset, profiles, profile

switching, profile editing/reactivation, user search, and NGO profile approval-state handling.

- **Identity:** User document submission, status display, administrative

review, and action logging.

- **Volunteering:** NGO opportunities, volunteer signup, profile documents,

attendance, event messages, certificates, search, and administrative export.

- **Blood donation:** Donor profiles/search, blood requests, matching,

completion/history, reports, and notifications.

- **Goods exchange:** Listings, interest, conversations, messaging, and reports.
- **Health:** Questions/replies and health-professional applications/review.
- **Farmer marketplace:** Produce listings, owner/admin editing of existing

listing fields, availability controls, and price-related views. Product images are not supported by the current listing model.

- **Waste services:** Pickup requests, collectors, ratings, and reviews.
- **Shared services:** Ratings, notifications, reporting, moderation, and

administration.

2.2 Requirement traceability limitation

No SRS, full user-story register, approved change record, or teacher sample report was present in the checkout. The feature list above is derived from source code, routes, tests, and existing project notes; it cannot be treated as a complete or approved requirements baseline.

An implementation of multiple profiles is visible in Git history, but the prior requirement, rationale, approval, and stakeholder impact are not documented. See the separate Requirement Change Log.

3. System analysis and design

3.1 User categories and access

The source includes citizen and NGO registration, volunteer profiles, NGO coordinator workflows, and superuser administration. Access is enforced using authentication, role/profile checks, object ownership, and administrative permissions in API views. Because the project retains a legacy user-role field alongside profiles, some role-sensitive workflows require product-policy review.

3.2 Architecture

HELPNET uses a server-rendered Django application:

1. A browser loads HTML templates and static JavaScript/CSS served by Django.
2. JavaScript calls Django REST Framework endpoints, commonly using JWT bearer

tokens.

3. API views validate requests and apply authentication/authorization.
4. Django models and migrations persist application data.
5. Local filesystem storage is used for uploads; sensitive documents are

served through permission-checked download handlers.

The shared frontend request helper handles bearer tokens; some pages also use direct `fetch` calls. Automatic token refresh was not found.

3.3 Data model overview

The main data areas include users and profiles; volunteer profiles, opportunities, signups, attendance, messages, and certificates; donor profiles, blood requests, and donation history; goods listings, interests, conversations, messages, and reports; health questions, replies, and professional applications; produce listings; waste collectors, pickup requests, and ratings; general ratings, notifications, reports, verification requests, and admin action logs.

Model-level foreign keys and uniqueness constraints support ownership and one-to-one or one-per-user/event relationships. A known limitation is that ratings are unique by rater and target user, not by a specific transaction; there is no interaction model to define transaction-level ratings.

3.4 Data protection design

Private identity, health credential, volunteer, and blood-request documents use private storage and authorized download handlers. During this audit, blood-request documents were changed from public media to private storage. Existing referenced documents are moved by a data migration. The blood document access policy implemented for this audit is requester/owner and administrator only.

4. Implementation

4.1 Technology

- Python 3.14 environment used for audit execution
- Django 5.2.17
- Django REST Framework and SimpleJWT
- MySQL configured for ordinary application runs
- SQLite selected for pytest
- Django templates, HTML, CSS, and vanilla JavaScript
- ReportLab is used by existing certificate generation

4.2 Application areas

The project is organized into Django apps under `backend/apps/`, with project settings and URL configuration in `backend/helpnet/`. Browser pages and static assets are under `frontend/pages/`, `frontend/js/`, and `frontend/css/`. Automated browser tests are stored separately under `tests/selenium/`.

4.3 Audit corrections

The audit made the following corrections in the current worktree:

1. Replaced source-hardcoded development settings with environment-backed

secret, DEBUG, host, CORS, and HTTPS-cookie configuration.

2. Removed user-controlled HTML interpolation from the verification admin

script by using safe DOM text.

3. Loaded the health-professional review script on the correct admin page.

4. Corrected the volunteer certificate collection POST to accept and validate

`event_id` from the request body.

5. Made blood-request uploads private, validated content signatures, restricted

document downloads, and added a data migration for existing files.

6. Returned null rather than a fabricated rating when no ratings exist, and

updated the relevant UI display.

7. Added ignore rules and removed nineteen private uploads from the current Git

index while preserving local copies.

8. Updated the root README and project summary with setup steps and current

limitations.

9. Added owner-scoped profile detail editing and reactivation, and separated

pending/rejected NGO approval status from the user's personal NID status.

10. Added owner/admin edits for supported farmer listing fields and availability.

11. Added targeted Bangla/English translations for dashboard role/activity text,

NGO approval status, and farmer editing; restyled the volunteer opportunity navigation control.

These are uncommitted changes. The sensitive-file deletions are staged in the index; other code and documentation changes are unstaged.

5. Testing and results

5.1 Backend tests

The final backend run passed **115 tests** across Blood, Common, Farmer, Goods, Health, Notifications, Ratings, Reports, Users, Verification, Volunteer, and Waste. Tests cover selected API validation, permissions, ownership, data transitions, document handling, ratings, and page-script wiring.

5.2 Additional checks

- Django system check under test/SQLite configuration: **passed**.
- Migration drift check: **passed; no model changes detected**.
- Targeted Python compilation: **passed**.
- `git diff --check`: **passed**.
- Selenium collection: **91 cases collected; not executed**.
- Production-mode configuration simulation: secure defaults verified; HSTS

remains unset and Django reports that as a deployment warning.

5.3 Limitations of testing

The 115 tests are backend tests and do not establish that all browser workflows work end to end. Selenium execution was unavailable because this environment has no browser/WebDriver and no running seeded application. JavaScript syntax validation was unavailable because Node.js/npm were not installed. MySQL-backed checks and a live migration were unavailable because `mysqlclient`/`MySQLdb` and a database connection were unavailable.

The repository includes an earlier Selenium results document, but it was not rerun during this audit and is not represented as a result of the current verification.

6. Deployment and operation

6.1 Local setup

Use the repository README and `backend/.env.example` for environment setup. Configure database settings and secret values outside source control, apply migrations, then start Django. The `.env` file and private media should remain untracked.

6.2 Production requirements

Before deployment, configure a strong random `SECRET_KEY`, `DEBUG=False`, explicit `ALLOWED_HOSTS`, a minimum CORS origin list, the actual MySQL connection, and the HTTPS/proxy settings appropriate to the deployment. Enable HSTS only after verifying the domain's HTTPS and subdomain policy. Apply migration `blood.0004` with access to legacy media and validate the resulting file counts and access controls.

6.3 Operational gaps

No automated backup/restore job or recovery drill was verified. Cloudinary is declared as a dependency but is not configured as active storage. Production MySQL behavior, actual HTTPS termination, email delivery, and upload migration have not been exercised against a deployed environment.

7. Known limitations and risks

- Previously committed private uploads and former development settings remain

in historical Git commits. Current-index removal does not purge history.

- The production HSTS decision depends on deployment topology.
- Donor search accepts an `available` parameter whose intended semantics are

undocumented; current behavior does not apply the parameter.

- Ratings are per rater/target rather than per service interaction.
- `GoodsReport` is modeled, while the report endpoint uses generic `Report`.
- Health-professional application eligibility uses the legacy Citizen role.
- Localization exists, but complete coverage was not acceptance-tested.
- NGO profile submission uses only the existing account information and

organization display name, as requested; NGO document upload and a separate NGO-details model are not implemented.

- Cloudinary integration and automated backup/restore are not implemented.
- Formal SRS/SDS, requirements traceability, team contribution evidence,

budget, approval records, and stakeholder acceptance are missing.

8. Team, schedule, and change control

Team member names, student IDs, individual contributions, supervisor, project schedule, budget, and stakeholder sign-offs were not confirmed by repository evidence. The team must complete these items rather than infer them from historical test rows or commit authors.

The separate requirement change log distinguishes the observed multi-profile implementation from a formally approved requirement change. No approval record was found. The blood-document access rule was explicitly confirmed during the audit as requester/owner and administrators only; the authority for final stakeholder acceptance should still be confirmed.

9. Conclusion and recommendations

HELPNET contains a broad set of community-service APIs and matching web interfaces. The latest backend suite passes, and the audit corrected several security/reliability defects and implemented the requested profile, farmer listing, NGO-status, and targeted localization updates. The project should not be described as fully production-verified until browser workflows, MySQL migrations, deployment settings, backup/restore, and stakeholder acceptance are tested.

Recommended next steps:

1. Have the project team validate the feature inventory and complete the missing requirements and contribution records.
2. Run Selenium tests against a seeded environment with a supported browser.
3. Validate migrations and private-file access on a staging MySQL/media copy.
4. Confirm HTTPS/HSTS, allowed hosts, CORS, and email configuration with the deployment owner.
5. Coordinate secret rotation and an approved Git-history cleanup with the repository owner.
6. Perform and document a backup/restore drill.
7. Record stakeholder acceptance and attach genuine, privacy-safe screenshots and a real demonstration recording.

Appendix A. Audit artifacts

- Project audit
- Bug tracker
- Code change log
- Test report
- Security report
- Execution log
- Final audit report

Appendix B. Sign-off

- **Team lead:** [Name / signature / date]
- **Supervisor:** [Name / signature / date]
- **Stakeholder/business owner:** [Name / acceptance / date]